

. Tạo tài khoản và đăng nhập

- Đăng ký/đăng nhập: dùng GitLab.com (Cloud) hoặc cài GitLab Self-Managed (CE/EE) trên máy chủ công ty.
- Tài khoản: đăng ký bằng email công ty hoặc dùng SSO (SAML/OAuth) của tổ chức để đảm bảo quản lý người dùng tập trung.
- Bảo mật tài khoản: bật 2FA cho mọi user; yêu cầu mật khẩu mạnh và chính sách mật khẩu qua SSO.

2. Cấu trúc nhóm (Group) và repository (Project)

- Tạo Group cho tổ chức hoặc dự án: ví dụ “booking-hotel” làm group chứa tất cả repo liên quan (backend, frontend, mobile, infra, docs).
- Trong Group, tạo Projects cho từng component: backend-api, web-frontend, mobile-app, infra-terraform, docs.
- Quy ước đặt tên: <component>-<type>-<env> (ví dụ: backend-api, infra-terraform).
- Sử dụng Subgroups để phân tầng (e.g., booking-hotel/backend, booking-hotel/frontend).

3. Quản lý quyền và thành viên

- Roles: Owner (quản trị group), Maintainer (merge/pipeline), Developer (push branches/merge request), Reporter, Guest.
- Gán quyền theo nhóm: tạo nhóm người dùng (dev-backend, dev-frontend, qa, devops, pm) và gán vào GitLab Group với role tương ứng.
- Protected branches: bảo vệ branch chính (main/master, release/*); chỉ cho phép Maintainer hoặc role quy định merge.
- Protected tags: hạn chế ai có thể tạo tag production.

4. Quy trình làm việc (Git workflow)

- Branch model đề xuất:
 - main (production-ready)
 - develop (tích hợp cho sprint) — tùy đội có thể dùng trunk-based thay cho develop
 - feature/<ticket-id>-short-description

- hotfix/<issue>
- release/<version>
- Tạo branch từ develop (hoặc main nếu trunk-based).
- Mỗi task/story/bug gắn MR (Merge Request) trỏ tới issue Jira/GitLab issue.
- Code review: bắt buộc review 1–2 approver, bắt buộc CI pipeline pass trước merge.
- Commit message convention: feat|fix|chore(scope): short description; reference issue id (#123).

5. Issues và Boards

- Sử dụng GitLab Issues để track tasks nếu không dùng Jira; nếu dùng Jira, link issue giữa GitLab và Jira.
- Issue template: tạo templates cho Bug, Feature, Incident, RFC (Request for change) để chuẩn hoá thông tin (mô tả, reproduction, acceptance criteria).
- Boards: tạo board theo workflow (To Do, In Progress, In Review, QA, Done) để visualize progress; swimlanes theo assignee hoặc milestone.

6. Merge Requests (MR)

- Mẫu MR: tiêu đề chuẩn [PROJECT-123] short summary; trong description thêm checklist, liên kết issue, testing steps, ảnh/screenshots.
- MR approvals: cấu hình số approvers, set approver groups (e.g., tech-lead, security).
- Pipelines required: cấu hình protected branch require pipeline success before merge.
- Merge method: Merge commit hoặc squash merge (tuỳ policy). Squash giúp giữ lịch sử gọn.

7. CI/CD với .gitlab-ci.yml

- File cấu hình: đặt .gitlab-ci.yml ở root project. Định nghĩa stages: prepare, build, test, lint, package, deploy.
- Shared templates: tái dùng templates/job via include từ group-level templates.
- Variables: sử dụng CI/CD variables cho secrets (MARK AS PROTECTED cho biến production).

- Environments & Jobs: define environments (staging, production) và jobs deploy tương ứng.
- GitLab Environments & Review Apps: bật review apps cho mỗi MR (tạo ephemeral preview env cho frontend hoặc full stack nếu có thể).
- Artifacts & Caching: lưu artifact build, cache dependencies để tăng tốc pipeline.

8. Runners (GitLab Runner)

- Managed runners vs Shared: dùng shared runners GitLab.com cho dự án nhỏ; cho dự án production dùng self-hosted runners (on-prem or cloud) để kiểm soát tài nguyên.
- Tags: gán tags cho runners (docker, linux, macos, mac-ios) và reference tags trong jobs.
- Security: chạy runners trong môi trường cách ly (docker executor, Kubernetes executor) và giới hạn quyền truy cập network.
- Autoscaling: dùng GitLab Runner autoscale (Kubernetes) để mở rộng theo tải pipeline.

9. Quản lý Secrets và Config

- CI/CD variables: lưu API keys, credentials, và đánh dấu protected để chỉ env production access.
- Avoid storing secrets in repo: dùng Vault, AWS Secrets Manager hoặc GitLab Secrets (Variables) kết hợp với masking cho sensitive.
- Config management: dùng IaC (Terraform) repo để quản lý infra; phân branch release/PR cho cấu hình.

10. Deploy strategy & Releases

- Release branches/tags: gán tag semantic versioning (v1.2.3) khi deploy production.
- Deployment strategies: Blue-Green, Canary, Rolling — cấu hình trong jobs deploy và phối hợp với Kubernetes/Helm.
- Release notes: tự động sinh release notes từ MR/commit messages (GitLab release feature).
- Rollback: maintain scripts/playbooks hoặc Helm rollback; ensure DB migration reversible or separate migration strategy.

11. Monitoring, Tracing & Observability

- Integrate GitLab with monitoring tools: Prometheus (GitLab has built-in support), Grafana dashboards.
- CI job metrics: monitor pipeline duration, failure rate, flaky tests; alert khi pipeline failed for protected branches.
- Deployments: link environment to monitoring and health checks; use readiness/liveness checks for Kubernetes.

12. Security, Scanning và Compliance

- Enable GitLab built-in security scans: SAST, DAST, Dependency Scanning, Container Scanning, License Compliance.
- Pipeline gates: block merges when critical vulnerabilities detected (configure security policies).
- Code owners: use CODEOWNERS file to auto-request reviews from responsible teams (e.g., backend owners for API changes).
- Audit logs: enable audit events (available on GitLab EE or GitLab.com with appropriate plan) to track changes, access and configurations.

13. Integrations hữu ích

- Jira: link commits/MR to Jira issues; two-way integration nếu dùng Jira Cloud.
- Slack / Microsoft Teams: notify channels on MR created/merged, pipeline failed, production deploys.
- Container registry: dùng built-in Container Registry để lưu docker images (per project).
- Package registry / NPM / Maven: store artifacts.
- Helm charts registry: store release charts for k8s deployments.
- Sentry/Datadog/PagerDuty: create alerts from production and link to MR/issue for triage.

14. Best practices và quy tắc nhóm

- CI must be green: không merge khi pipeline fail.
- Small, focused MRs: giảm scope MR để review nhanh và giảm rủi ro.

- Enforce code review: 1–2 approvers; use CODEOWNERS.
- Test coverage: chạy unit, integration và e2e trong pipeline; giữ regression suite tối ưu để không làm chậm CI quá nhiều.
- Use feature flags: triển khai code off by default và bật dần bằng feature flag cho release an toàn.
- Document everything: README, contribution guidelines, pipeline README, deploy runbooks.

15. Onboarding checklist cho project repo

1. Tạo Project & set visibility (private).
2. Add members & groups with proper roles.
3. Create branch protections for main/release branches.
4. Add .gitlab-ci.yml mẫu và test pipeline.
5. Register runners (self-hosted) with proper tags.
6. Configure CI/CD variables (protected for production).
7. Create CODEOWNERS, ISSUE_TEMPLATE, MERGE_REQUEST_TEMPLATE.
8. Enable Container Registry and Package Registry if cần.
9. Setup integrations: Jira, Slack/Teams, monitoring.
10. Document contribution guide & running local dev instructions.

16. Các lỗi phổ biến và cách khắc phục nhanh

- Pipeline quá chậm: bật cache, split jobs, chạy tests song song, loại bỏ bước thừa.
- Runner không nhận job: kiểm tra tags match job tags, runner status, network access.
- Secrets lộ trong logs: enable masked variables, rotate compromised keys, remove secrets from commit history.
- MR blocked by security policy: kiểm tra report SAST/DAST, fix high vulnerabilities hoặc create exception process.

17. Tài liệu mẫu để copy-paste (cần thiết)

- .gitlab-ci.yml template cơ bản (build/test/deploy) — tôi có thể cung cấp mẫu cụ thể cho stack Node.js + Docker + Kubernetes nếu bạn muốn.
- Merge Request template: checklist, testing steps, linked issues.
- Issue templates: Bug, Feature, Incident.
- CODEOWNERS file mẫu.